DIGAIPE

Manual de instalación

Instalación completa con Docker o Podman
e instalación en servidor físico (Linux y Windows)

Versión del código: commit `a587434`
Fecha del manual: 10 de noviembre de 2026

Índice general

1. Introducción **4**

1.1. Propósito 4

1.2. Componentes 4

1.3. ¿Docker o servidor físico? 5

1.4. Convenciones de este manual 6

2. Requisitos y planificación **7**

2.1. Servidor 7

2.2. Sistemas operativos 7

2.3. Red 8

2.4. Zona horaria 8

2.5. Contraseñas y secretos 8

2.6. Hoja de datos de la instalación 9

3. Obtener el código **10**

3.1. Instalar Git 10

3.2. Clonar el repositorio 10

4. Instalación completa con Docker o Podman **12**

4.1. Instalar Docker 12

4.1.1. Linux (recomendado) 12

4.1.2. Windows 13

4.2. Alternativa: Podman sin root (Linux) 14

4.2.1. Requisitos 14

4.2.2. Instalar Podman y docker-compose 14

4.2.3. Usuario, puertos y carpetas 14

4.2.4. Construir y arrancar 15

4.2.5. Arranque automático 15

4.2.6. IP real de los usuarios 16

4.2.7. Otras diferencias con Docker 16

4.3. Obtener el código . . . 17
4.4. Configurar `.env` . . . 17
4.5. Construir y arrancar . . . 18
4.6. Cortafuegos . . . 18
4.7. Arranque automático . . . 19
4.8. Siguiente paso . . . 19
4.9. Operación diaria . . . 19

5. Instalación en servidor físico: Linux 20
5.1. Preparar el sistema . . . 20
5.2. Instalar Node.js 24 . . . 21
5.3. Instalar MySQL 8.4 . . . 21
5.4. Instalar la aplicación . . . 22
5.5. Archivos de configuración . . . 22
5.6. Comandos de administración . . . 24
5.7. Ejecutar la instalación . . . 24
5.8. Servicio de la aplicación . . . 25
5.9. Caddy (HTTPS) . . . 26
5.10. Servicio de respaldos . . . 26
5.11. Cortafuegos . . . 27
5.12. Siguiente paso . . . 27
5.13. Operación diaria . . . 27

6. Instalación en servidor físico: Windows 29
6.1. Preparar el sistema . . . 29
6.2. Instalar Node.js 24 . . . 30
6.3. Instalar MySQL 8.4 . . . 30
6.4. Instalar la aplicación . . . 31
6.5. Archivos de configuración . . . 31
6.6. Scripts de administración . . . 33
6.7. Ejecutar la instalación . . . 34
6.8. Servicio de la aplicación . . . 34
6.9. Caddy (HTTPS) . . . 35
6.10. Tarea programada de respaldos . . . 36
6.11. Cortafuegos . . . 36
6.12. Siguiente paso . . . 36
6.13. Operación diaria . . . 37

7. Certificado HTTPS en las PCs cliente **38**
7.1. Exportar la raíz . . . 38
7.2. Instalar la raíz en las PCs . . . 39
7.3. Conservar la autoridad . . . 39
7.4. Certificado público (Let's Encrypt) . . . 39
7.5. Usar el certificado de la institución . . . 40
7.6. Cambiar el nombre del servidor . . . 40

8. Verificación posterior a la instalación **41**

9. Respaldos y restauración **43**
9.1. Cómo funcionan . . . 43
9.2. Revisión semanal (5 minutos) . . . 43
9.3. Respaldo manual . . . 44
9.4. Antes de restaurar . . . 44
9.5. Restaurar . . . 44
9.6. Recuperar en un servidor nuevo . . . 45
9.7. Simulacro de restauración . . . 45

10. Auditoría: verificar y anclar **47**
10.1. Comandos . . . 47
10.2. Cuenta de solo lectura para auditores . . . 48

11. Actualización **49**
11.1. Docker . . . 49
11.2. Servidor físico . . . 49
11.3. Volver a la versión anterior . . . 50

12. Solución de problemas **51**

A. Referencia **54**
A.1. Variables de configuración . . . 54
A.2. Ubicaciones . . . 55
A.3. Cuentas de MySQL . . . 55
A.4. Roles de la aplicación . . . 56

Capítulo 1

Introducción

1.1. Propósito

Este manual explica cómo instalar DIGAIPE en un servidor de la red institucional, de principio a fin, de dos maneras:

- **Con contenedores** (capítulo 4): todos los componentes se ejecutan en contenedores definidos en `deploy/compose.yaml`, con Docker o con Podman ejecutado por un usuario sin privilegios (sección 4.2). En este manual, "Docker" designa este modo con cualquiera de los dos.
- **En servidor físico** (sin Docker), en Linux (capítulo 5) o en Windows (capítulo 6): cada componente se instala como un servicio del sistema operativo.

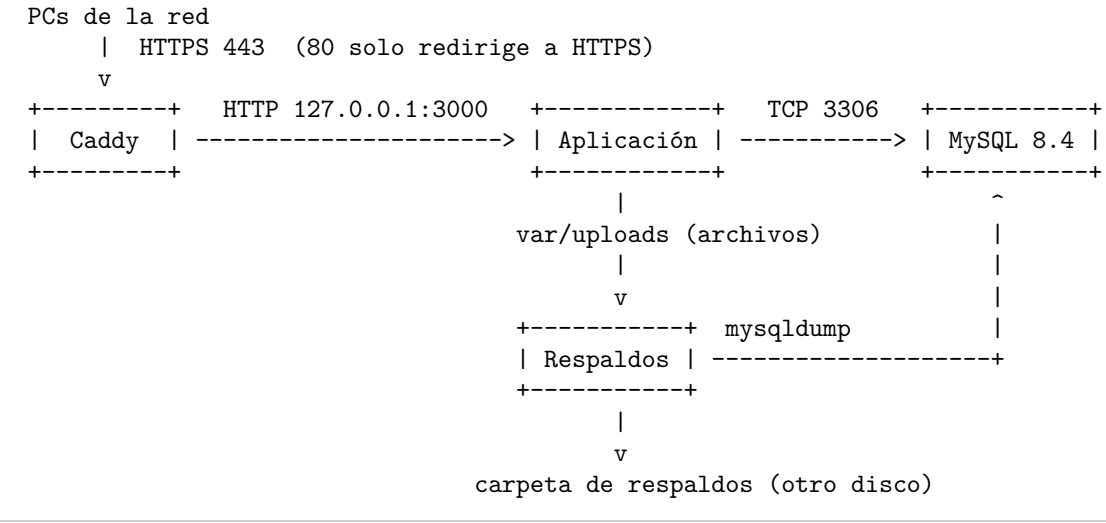
Ambas instalaciones dan el mismo resultado: la misma base de datos, las mismas cuentas con permisos mínimos, HTTPS con una autoridad certificadora local, respaldos semanales y un registro de auditoría verificable. El servidor descarga lo que necesita (código, paquetes e imágenes) directamente de internet durante la instalación y las actualizaciones.

1.2. Componentes

Componente	Función	Implementación
Base de datos	Datos de admisiones, usuarios, sesiones y registro de auditoría	MySQL 8.4 LTS
Aplicación	Interfaz web y lógica de negocio	Node.js 24 (SvelteKit, `node build`), escucha solo en el propio servidor, puerto 3000
Proxy HTTPS	Única entrada desde la red. Emite y renueva el certificado solo	Caddy 2 con autoridad certificadora local (`local_certs`) o, si el servidor es público, Let's Encrypt
Instalación (*setup*)	Migraciones, triggers de auditoría, cuentas de MySQL, administrador inicial y datos base. Se ejecuta y termina	`scripts/docker/setup.sh` y los scripts de `scripts/`

Componente	Función	Implementación
Respaldos	Copia semanal de la base y de los archivos cargados, con suma SHA-256	`deploy/backup/*.sh` (bash, `mysqldump`)
Auditoría	Verificación y anclaje de la cadena de hashes	`scripts/audit-ledger.ts`

Flujo de red (igual en ambos modos)



Ni MySQL (3306) ni la aplicación (3000) aceptan conexiones desde la red: solo Caddy publica puertos. Esto no es opcional; la aplicación confía en la cabecera `X-Forwarded-For` que añade Caddy para registrar la dirección IP real de cada usuario en la auditoría. Si la aplicación fuera accesible directamente, cualquiera podría falsificar esa dirección.

1.3. ¿Docker o servidor físico?

	Docker	Servidor físico
Ventaja principal	Es la instalación que mantiene el repositorio (`deploy/`); un solo archivo de configuración; instalar y actualizar son dos o tres comandos	No requiere Docker; funciona en Windows Server; cada pieza se ve y se administra con las herramientas habituales del sistema
Desventaja	Requiere Docker. En Windows, Docker Desktop tiene limitaciones serias para un servidor (sección 4.1.2)	Más pasos, más archivos de configuración; las actualizaciones se hacen a mano
Respaldos	Servicio `backup-worker` incluido	Servicio `systemd` (Linux) o tarea programada (Windows)
Dificultad	Baja	Media

Recomendación

Use Docker sobre Linux siempre que pueda: es la configuración probada por el equipo de desarrollo y la que menos pasos manuales tiene. Use la instalación en servidor físico cuando la política institucional no permita Docker o cuando el servidor disponible sea Windows Server.

1.4. Convenciones de este manual

- **Linux** Los comandos se ejecutan en una terminal con un usuario que puede usar `sudo`.
- **Windows** Los comandos se ejecutan en **PowerShell como administrador** (clic derecho → Ejecutar como administrador), salvo que se indique otra cosa.
- Lo que va entre `<` y `>` se reemplaza por su valor, sin los signos. Por ejemplo, `<DIGAIPE_HOST>` se reemplaza por `digaipe.ula.local`.
- Las rutas son literales y las mismas en todo el manual. Si elige otras, cámbielas en *todos* los archivos y comandos. El apéndice A.2 las resume.

Datos personales

La base de datos, la carpeta de archivos cargados y los respaldos contienen datos de estudiantes, hashes de contraseñas y sesiones. Los archivos de configuración contienen las contraseñas de MySQL. Trátelos todos como información confidencial: no los envíe por correo, no los suba a servicios externos y no los deje en carpetas compartidas sin control de acceso.

Capítulo 2

Requisitos y planificación

2.1. Servidor

Los valores siguientes son una recomendación para una oficina con decenas de usuarios simultáneos, no una medición. Ajústelos si el volumen de admisiones es mucho mayor.

Recurso	**Recomendación**
Procesador	2 núcleos de 64 bits (x86-64)
Memoria	4 GB (8 GB si usa Docker Desktop en Windows)
Disco del sistema	40 GB libres para sistema, programas, base de datos y archivos cargados
Disco de respaldos	Un **segundo disco**, con al menos 12 veces el tamaño de un respaldo. Un respaldo en el mismo disco que la base de datos se pierde junto con ella
Red	Dirección IP fija o un nombre en el DNS interno

2.2. Sistemas operativos

Modo	**Linux**	**Windows**
Docker	Debian 12 o Ubuntu 24.04 LTS con Docker Engine y el complemento `docker compose`	Windows 10/11 Pro o Enterprise con Docker Desktop (WSL 2). Vea las limitaciones en la sección 4.1.2
Podman	Debian 13, Fedora o RHEL 9.5 o posterior (Podman 5) con el binario `docker-compose` v2 (sección 4.2)	No recomendado
Servidor físico	Debian 12 o Ubuntu 24.04 LTS	Windows Server 2022 o 2025 (también Windows 10/11 Pro)

Para el servidor físico se necesita, además:

- **MySQL 8.4 LTS**. Es la versión que usan las imágenes Docker. No se ha probado con MySQL 8.0, con MySQL 9 ni con MariaDB: no use otra versión sin probarla antes en un equipo aparte.

- **Node.js 24** (la misma versión mayor que la imagen `node:24`).
- **Caddy 2**.
- Solo en Windows: **Git for Windows** (aporta `bash`, necesario para los scripts de respaldo) y **NSSM** (ejecuta la aplicación como servicio de Windows).

2.3. Red

- Elija el **nombre o la IP** con que las PCs abrirán el sistema, por ejemplo `digaipe.ula.local` o `192.168.1.50`. En este manual se llama `<DIGAIPE_HOST>`. El certificado HTTPS se emite para ese valor exacto: si cambia después, hay que volver a configurar (sección 7.6).
- Abra en el cortafuegos del servidor **solo** los puertos TCP 80 y 443.
- **No** abra los puertos 3306 (MySQL) ni 3000 (aplicación).
- Durante la instalación y las actualizaciones, el servidor necesita salida a internet por HTTPS hacia: `github.com` (código), `registry.npmjs.org` y `cdn.sheetjs.com` (paquetes de npm), los repositorios del sistema operativo y, según el modo, Docker Hub (`registry-1.docker.io`) o los sitios de descarga de Node.js, MySQL y Caddy. Si la red usa un proxy, configúrelo antes (variables `HTTPS_PROXY` y `HTTP_PROXY`, y `npm config set proxy`).

2.4. Zona horaria

El sistema usa la hora de Caracas (`America/Caracas`, UTC−4 todo el año). Los contenedores la fijan solos; en servidor físico se configura en el sistema operativo (capítulos 5 y 6). Los respaldos se programan en UTC: `BACKUP_HOUR_UTC=6` equivale a las 02:00 de Caracas.

2.5. Contraseñas y secretos

La instalación necesita los siguientes secretos. Genérelos antes de empezar y guárdelos en un gestor de contraseñas o en un sobre cerrado bajo custodia, según la política de la institución.

Secreto	Uso
`MYSQL_ROOT_PASSWORD`	Dueño de la base de datos. Solo para instalar, actualizar, restaurar y verificar la auditoría
`APP_DB_PASSWORD`	Cuenta de MySQL de la aplicación (permisos mínimos)
`BACKUP_DB_PASSWORD`	Cuenta de MySQL de los respaldos (solo lectura y anotar eventos `backup.*`)
`READONLY_DB_PASSWORD`	Opcional. Cuenta de solo lectura para auditores
`BETTER_AUTH_SECRET`	Firma de las sesiones. 32 caracteres aleatorios o más
`BOOTSTRAP_ADMIN_PASSWORD`	Contraseña del primer administrador de la aplicación

Solo letras y números

Las contraseñas de MySQL van dentro de una URL de conexión (`mysql://usuario:contraseña@servidor/base`). Un carácter como `@`, `:`, `/`, `#` o `%` rompe la URL. Use **solo letras y números**. Los comandos siguientes generan 48 caracteres hexadecimales, que cumplen esta regla y sirven para todos los secretos.

Linux

```
openssl rand -hex 24
```

Windows

```
$b = New-Object byte[] 24
[Security.Cryptography.RandomNumberGenerator]::Create().GetBytes($b)
-join ($b | ForEach-Object { $_.ToString('x2') })
```

Ejecute el comando una vez por cada secreto. **No use** los valores de ejemplo de `deploy/.env.example` (por ejemplo, `BOOTSTRAP_ADMIN_PASSWORD=123456`).

2.6. Hoja de datos de la instalación

Complete esta hoja antes de empezar y archívela con el registro de la instalación. No escriba aquí las contraseñas.

Dato	Valor
Modo de instalación (Docker / físico Linux / físico Windows)	
Nombre del servidor y sistema operativo	
`DIGAIPE_HOST` (nombre o IP para las PCs)	
Correo del primer administrador	
Carpeta de respaldos (disco)	
Medio externo para copias semanales de respaldos	
Medio del archivo de anclas de auditoría (no modificable por el DBA)	
Custodio de las contraseñas	
SHA-256 del certificado raíz HTTPS	
Fecha, responsable y versión (commit) instalada	

Capítulo 3

Obtener el código

Todos los modos parten de una copia del repositorio en el servidor, siempre en la misma carpeta:

Servidor	Carpeta del código
Linux	`/opt/digaipe/app`
Windows	`C:\digaipe\app`

En Docker, `docker compose` se ejecuta desde la subcarpeta `deploy` de esa copia; en servidor físico, la aplicación se ejecuta directamente desde ella.

3.1. Instalar Git

Linux

```
sudo apt update
sudo apt install -y git
```

Windows Descargue e instale Git for Windows desde `https://git-scm.com/download/win` con las opciones por defecto, o, si el servidor tiene `winget`:

```
winget install --id Git.Git -e --source winget
```

Cierre y vuelva a abrir PowerShell para que se actualice la variable `PATH`.

3.2. Clonar el repositorio

Linux

```
sudo mkdir -p /opt/digaipe
sudo git clone https://github.com/luishok/digaipe.git /opt/digaipe/app
cd /opt/digaipe/app
sudo git checkout <RAMA_O_ETIQUETA_A_INSTALAR>
sudo git log -1 --format="%h %cd"
```

La copia pertenece a `root`; por eso todos los comandos `git` sobre ella llevan `sudo` (sin él, Git se niega con *detected dubious ownership*).

Windows

```
New-Item -ItemType Directory -Force C:\digaipe
git clone -c core.autocrlf=false https://github.com/luishok/digaipe.git
↪  C:\digaipe\app
Set-Location C:\digaipe\app
git checkout <RAMA_O_ETIQUETA_A_INSTALAR>
git log -1 --format="%h %cd"
```

Si el repositorio es privado, Git pedirá usuario y un *token* de acceso de GitHub (no la contraseña de la cuenta). Use un token de solo lectura limitado a este repositorio.

Anote en la hoja de datos el commit que muestra el último comando.

Finales de línea

Los scripts `.sh` deben tener finales de línea LF; con CRLF fallan con `$'\r': command not found`. En Windows, Git convierte por defecto a CRLF: por eso se clona con `-c core.autocrlf=false`. Compruébelo en ambos sistemas:

```
git ls-files --eol deploy/backup/ scripts/docker/
```

(en Linux, con `sudo` delante) La segunda columna debe decir `w/lf` en todos los archivos.

Nota

La copia del servidor es solo para instalar: no edite archivos dentro de ella. La configuración (contraseñas) va en archivos que Git ignora (`deploy/.env`) o fuera de la carpeta (servidor físico), y los archivos cargados están en `var/uploads`, que Git también ignora. Así, actualizar es un `git pull` sin conflictos (capítulo 11).

Capítulo 4

Instalación completa con Docker o Podman

En este modo, `deploy/compose.yaml` define todos los servicios. El mismo archivo funciona con Docker y con Podman ejecutado por un usuario sin privilegios (sección 4.2).

Podman

Con Podman, **donde este manual dice `docker compose`, escriba `podman compose`**, como usuario `digaipe` y desde la carpeta `deploy`. Lo mismo para `docker build`, `pull`, `save` y `load`. Los pasos de instalación propios de Podman están en la sección 4.2.

Servicio	Qué hace
`db`	MySQL 8.4. No publica puertos
`setup`	Migraciones, triggers de auditoría, cuentas de MySQL, administrador inicial y datos base. Se ejecuta en cada arranque y termina; todos sus pasos se pueden repetir sin riesgo
`app`	La aplicación. Arranca solo si `setup` terminó bien. No publica puertos
`caddy`	HTTPS en los puertos 80 y 443
`backup-worker`	Respaldo semanal automático
`backup`, `restore`, `audit`	Perfil `ops`: se ejecutan a mano, nunca con `up`

Las imágenes `app` y `setup` se construyen en el propio servidor a partir del código; `mysql` y `caddy` se descargan de Docker Hub.

4.1. Instalar Docker

4.1.1. Linux (recomendado)

Docker Engine con el complemento Compose, desde el repositorio oficial de Docker. En **Debian 12**:

```
sudo apt update
```

```
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/debian/gpg -o
↪  /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture)
↪  signed-by=/etc/apt/keyrings/docker.asc]
↪  https://download.docker.com/linux/debian $(. /etc/os-release && echo
↪  "$VERSION_CODENAME") stable" | sudo tee /etc/apt/sources.list.d/docker.list >
↪  /dev/null
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
↪  docker-compose-plugin
sudo systemctl enable --now docker
```

En **Ubuntu 24.04**, los mismos comandos cambiando `linux/debian` por `linux/ubuntu` en las dos direcciones. No use el paquete `docker.io` de la distribución: suele traer una versión de Compose antigua.

Comprobación

```
sudo docker version
sudo docker compose version
```

Ambos deben responder sin error. En adelante, los comandos `docker` se muestran sin `sudo`; antepóngalo si su usuario no pertenece al grupo `docker`.

Nota

Pertenecer al grupo `docker` equivale a tener acceso de administrador al servidor. Agregue solo a quienes administran el sistema.

4.1.2. Windows

En Windows, Docker funciona mediante **Docker Desktop** con WSL 2 (Windows 10/11 Pro o Enterprise). Antes de elegir esta opción, tenga en cuenta:

- **Docker Desktop solo funciona con un usuario conectado.** Si nadie inicia sesión en el servidor después de un reinicio, DIGAIPE no arranca.
- **Licencia.** Docker Desktop requiere una suscripción de pago en organizaciones grandes (más de 250 empleados o más de 10 millones de dólares de ingresos anuales). Consúltelo con la institución antes de instalarlo.
- No es compatible con Windows Server.

Recomendación

En un servidor Windows, use la instalación en servidor físico (capítulo 6), o una máquina virtual Linux con Docker Engine.

Si de todos modos usa Docker Desktop: instálelo desde `https://www.docker.com`, habilite WSL 2 cuando lo pida, reinicie, y en *Settings* → *General* active *Start Docker Desktop when you sign*

in. Compruebe en PowerShell con `docker version` y `docker compose version`.

4.2. Alternativa: Podman sin root (Linux)

Podman ejecuta los contenedores con un usuario normal, sin un demonio con privilegios de `root`. Si alguien toma el control de un contenedor, queda limitado a ese usuario. Si usa Docker, salte esta sección.

4.2.1. Requisitos

- **Podman 5** o posterior: Debian 13, Fedora, o RHEL 9.5 o posterior. Debian 12 y Ubuntu 24.04 traen Podman 4, que no se ha probado con esta instalación.
- El binario **`docker-compose` v2**. `podman compose` lo usa cuando lo encuentra. **No** use `podman-compose`: no admite bien las condiciones `service_healthy` y `service_completed_successfully`, los perfiles ni las anclas YAML de `compose.yaml`.
- Un usuario dedicado, `digaipe`, que ejecuta todos los contenedores. Es una cuenta normal, distinta de la cuenta de servicio del capítulo 5.

4.2.2. Instalar Podman y docker-compose

En **Debian 13** (en Fedora o RHEL, `sudo dnf install -y podman`):

```
sudo apt update
sudo apt install -y podman systemd-container
```

Descargue `docker-compose-linux-x86_64` y `docker-compose-linux-x86_64.sha256` de la versión más reciente en `https://github.com/docker/compose/releases`, compruebe la suma e instálelo:

```
sha256sum -c docker-compose-linux-x86_64.sha256
sudo install -m 755 docker-compose-linux-x86_64 /usr/local/bin/docker-compose
```

Comprobación

`podman --version` debe mostrar `5.<x>` o posterior, y `docker-compose version`, `v2.<x>`.

4.2.3. Usuario, puertos y carpetas

```
sudo useradd --create-home --shell /bin/bash digaipe
grep digaipe /etc/subuid /etc/subgid
sudo loginctl enable-linger digaipe
echo 'net.ipv4.ip_unprivileged_port_start=80' | sudo tee
↪  /etc/sysctl.d/99-digaipe.conf
sudo sysctl --system
```

- `grep` debe mostrar un rango para `digaipe` en ambos archivos: son los identificadores que usan los contenedores. Si no aparece, agréguelo con `sudo usermod --add-subuids 100000-165535 --add-subgids 100000-165535 digaipe`.
- `enable-linger` hace que los servicios del usuario funcionen sin que nadie inicie sesión, también después de un reinicio.
- Un usuario normal no puede abrir puertos menores de 1024; el `sysctl` permite abrir desde el 80, que Caddy necesita (80 y 443).

Haga el resto de la sección "Configurar `.env`" de este capítulo como se indica, y luego entregue al usuario `digaipe` el archivo y las carpetas que usan los contenedores (ajuste las rutas a las que puso en `BACKUP_DIR` y `ANCHOR_DIR`):

```
sudo chown digaipe:digaipe /opt/digaipe/app/deploy/.env
sudo chmod 600 /opt/digaipe/app/deploy/.env
sudo mkdir -p /srv/respaldos/digaipe /srv/anclas/digaipe
sudo chown digaipe:digaipe /srv/respaldos/digaipe /srv/anclas/digaipe
```

El código en `/opt/digaipe/app` sigue perteneciendo a `root`: el usuario `digaipe` solo necesita leerlo.

4.2.4. Construir y arrancar

Entre como `digaipe` con `machinectl`. **No** use `sudo -u` ni `su`: no crean la sesión de `systemd` que necesitan el socket de Podman y los healthchecks, y `setup` y `app` nunca arrancarían.

```
sudo machinectl shell digaipe@
systemctl --user enable --now podman.socket
cd /opt/digaipe/app/deploy
podman compose build
podman compose up -d
```

Compruebe igual que con Docker (sección "Construir y arrancar" más adelante), con `podman compose ps -a` y `podman compose logs setup`. El aviso `>>>> Executing external compose provider` es normal. Para ocultarlo, agregue `compose_warning_logs = false` en la sección `[engine]` de `~/.config/containers/containers.conf`.

4.2.5. Arranque automático

Podman no tiene un demonio que vuelva a arrancar los contenedores después de reiniciar el servidor. Como usuario `digaipe`, cree este servicio de usuario:

/home/digaipe/.config/systemd/user/digaipe.service

```
[Unit]
Description=DIGAIPE (podman compose)
Wants=podman.socket
After=podman.socket

[Service]
Type=oneshot
```

```
RemainAfterExit=yes
WorkingDirectory=/opt/digaipe/app/deploy
ExecStart=/usr/bin/podman compose up -d
ExecStop=/usr/bin/podman compose stop
TimeoutStartSec=600

[Install]
WantedBy=default.target
```

```
mkdir -p ~/.config/systemd/user
nano ~/.config/systemd/user/digaipe.service
systemctl --user daemon-reload
systemctl --user enable digaipe.service
```

Comprobación

Reinicie el servidor una vez. Sin iniciar sesión gráfica, entre con `sudo machinectl shell digaipe@` y compruebe con `podman compose ps -a` que todo volvió a arrancar.

4.2.6. IP real de los usuarios

La auditoría y el límite de intentos de inicio de sesión usan la IP que recibe Caddy. Según la versión y la red de Podman sin root, Caddy puede ver la misma dirección interna (`10.x`, `169.254.x`) para todas las conexiones. La auditoría perdería entonces la IP real, y un usuario que se equivoca de contraseña bloquearía a todos.

Compruébelo antes de usar el sistema

Inicie sesión desde **otra PC** y abra Administración → Auditoría: el evento debe mostrar la IP de esa PC (punto 6 del capítulo 8). Si muestra una dirección interna, no use esta instalación: cambie a Podman con `root` (`sudo podman compose ...`, con el servicio del sistema `podman.socket`) o a Docker.

4.2.7. Otras diferencias con Docker

- **Cortafuegos.** Podman sin root no modifica las reglas del sistema (Docker sí): `ufw` o `firewalld` se aplican a los puertos 80 y 443 como a cualquier programa. Abra solo esos dos (sección “Cortafuegos”).
- **SELinux** (Fedora, RHEL). Las carpetas montadas llevan `:z` en `compose.yaml` para que los contenedores puedan leerlas. Si `BACKUP_DIR` está en un disco sin etiquetas SELinux (USB con FAT, NFS) y el montaje falla, quite `:z` de esa línea y monte el disco con la opción `context=`.
- **Volúmenes.** Quedan en `/home/digaipe/.local/share/containers/storage/volumes/`. Sus archivos pertenecen a identificadores del rango de `/etc/subuid`. Para copiarlos, use `podman volume export` o `podman unshare cp -a`.
- **Imágenes en otra PC.** `podman save` necesita `-m` para guardar varias imágenes en un mismo archivo.

4.3. Obtener el código

Clone el repositorio en `/opt/digaipe/app` (Linux) o `C:\digaipe\app` (Windows) siguiendo el capítulo 3. Todos los comandos `docker compose` de este manual se ejecutan **desde la carpeta `deploy`** de esa copia: `/opt/digaipe/app/deploy` o `C:\digaipe\app\deploy`.

4.4. Configurar `.env`

Linux

```
cd /opt/digaipe/app/deploy
sudo cp .env.example .env
sudo chmod 600 .env
sudo nano .env
```

Windows

```
Set-Location C:\digaipe\app\deploy
Copy-Item .env.example .env
icacls .env /inheritance:r /grant:r "*S-1-5-32-544:F" "*S-1-5-18:F"
notepad .env
```

El comando `icacls` deja el archivo accesible solo para los administradores (`S-1-5-32-544`) y el sistema (`S-1-5-18`). Se usan los identificadores en lugar de los nombres porque los nombres cambian con el idioma de Windows.

Complete los valores:

Variable	Valor
`MYSQL_ROOT_PASSWORD`	Secreto generado (sección 2.5)
`APP_DB_USER`	Dejar `digaipe_app`
`APP_DB_PASSWORD`	Secreto generado
`BACKUP_DB_USER`	Dejar `digaipe_respaldo`
`BACKUP_DB_PASSWORD`	Secreto generado
`READONLY_DB_PASSWORD`	Vacío, salvo que los auditores necesiten acceso SQL
`DIGAIPE_HOST`	Nombre o IP, sin `https://` ni puerto. Ej.: `digaipe.ula.local`
`ORIGIN`	`https://` seguido exactamente de `DIGAIPE_HOST`. Ej.: `https://digaipe.ula.local`
`DIGAIPE_VERSION`	Dejar `local`
`BETTER_AUTH_SECRET`	Secreto generado
`BOOTSTRAP_ADMIN_EMAIL`	Correo del primer administrador
`BOOTSTRAP_ADMIN_PASSWORD`	Secreto generado o contraseña fuerte. **No** deje `123456`
`BACKUP_DIR`	Carpeta de respaldos, idealmente en otro disco. Ej.: `/srv/respaldos/digaipe` o `D:/digaipe-respaldos`

Variable	Valor
`ANCHOR_DIR`	Carpeta del archivo de anclas, en un medio que el administrador de la base no pueda modificar (capítulo 10)

Las demás variables (frecuencia y conservación de respaldos) tienen valores por defecto adecuados; están descritas en el apéndice A.1.

4.5. Construir y arrancar

```
docker compose build
docker compose up -d
```

`build` construye las imágenes `digaipe/app` y `digaipe/setup` (descarga los paquetes de npm; tarda unos minutos la primera vez). `up -d` descarga MySQL y Caddy y arranca todo: MySQL inicializa la base, `setup` hace la instalación y luego arranca `app`.

Comprobación

```
docker compose ps -a
docker compose logs setup
```

- `db`, `app`, `caddy` y `backup-worker` deben estar `running` (y `db`, `healthy`).
- `setup` debe estar `exited (0)`.
- El registro de `setup` debe mostrar, en orden, `==> Migraciones`, `==> Triggers de auditoría`, `==> Usuarios de base de datos y permisos`, `==> Administrador inicial` y `==> Datos base`, sin errores.

Si `setup` terminó con error, `app` no arranca. Lea el mensaje en `docker compose logs setup`, corrija `.env` y vuelva a ejecutar `docker compose up -d`. Vea también el capítulo 12.

4.6. Cortafuegos

Linux Con `ufw`:

```
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
```

Omita la regla 22 si no administra el servidor por SSH. Docker inserta sus propias reglas para los puertos publicados; por eso `compose.yaml` solo publica los de Caddy.

Windows

```
New-NetFirewallRule -DisplayName "DIGAIPE HTTP/HTTPS" -Direction Inbound -Protocol
↪  TCP -LocalPort 80,443 -Action Allow
```

4.7. Arranque automático

Todos los servicios tienen `restart: unless-stopped`: vuelven a arrancar con Docker tras un reinicio del servidor. En Linux, basta con que el servicio `docker` esté habilitado (`systemctl enable docker`, hecho arriba). En Windows, vea las limitaciones de la sección 4.1.2. Con Podman, la política de reinicio no basta: configure el servicio de usuario de la sección 4.2.5.

4.8. Siguiente paso

Continúe con el certificado HTTPS (capítulo 7) y la verificación (capítulo 8).

4.9. Operación diaria

Tarea	Comando (desde la carpeta `deploy`)
Ver estado	`docker compose ps -a`
Ver registros de la aplicación	`docker compose logs -f app`
Ver la instalación	`docker compose logs setup`
Reaplicar la instalación (tras cambiar `.env`)	`docker compose up -d`
Detener	`docker compose down`
Iniciar	`docker compose up -d`

No use `down -v`

`docker compose down` conserva los datos. `docker compose down -v` **borra la base de datos, los archivos cargados y la autoridad certificadora**. No lo use salvo que quiera empezar desde cero y tenga un respaldo verificado.

Los datos viven en estos volúmenes de Docker:

Volumen	Contenido
`digaipe_db-data`	Base de datos
`digaipe_uploads`	Archivos de admisiones cargados
`digaipe_caddy-data`	Autoridad certificadora local y certificados
`digaipe_caddy-config`	Configuración interna de Caddy

Con Podman sin root, los volúmenes tienen los mismos nombres y están en `/home/digaipe/.local/share/containers/storage/volumes/`.

Capítulo 5

Instalación en servidor físico: Linux

Instalación sin Docker en Debian 12 o Ubuntu 24.04 LTS. Cada componente queda como un servicio de `systemd`. Todo se instala desde los repositorios oficiales de cada programa.

Ruta	Contenido
`/opt/digaipe/app`	Aplicación (propiedad de `root`, solo lectura para el servicio)
`/opt/digaipe/app/var/uploads`	Archivos de admisiones cargados (escritura del servicio)
`/etc/digaipe/`	Archivos de configuración con contraseñas (solo `root`)
`/var/backups/digaipe`	Respaldos (mejor en otro disco montado ahí)
`/usr/local/sbin/digaipe-*`	Comandos de administración

5.1. Preparar el sistema

Zona horaria, usuarios de servicio y carpetas:

```
sudo timedatectl set-timezone America/Caracas
sudo useradd --system --home-dir /opt/digaipe --shell /usr/sbin/nologin digaipe
sudo useradd --system --no-create-home --shell /usr/sbin/nologin --groups digaipe
↪  digaipe-respaldo
sudo mkdir -p /opt/digaipe /etc/digaipe /var/backups/digaipe
sudo chmod 700 /etc/digaipe
sudo chown digaipe-respaldo:digaipe-respaldo /var/backups/digaipe
sudo chmod 700 /var/backups/digaipe
```

- `digaipe` ejecuta la aplicación.
- `digaipe-respaldo` ejecuta los respaldos. Pertenece al grupo `digaipe` para poder *leer* los archivos cargados, pero no puede modificarlos, y la aplicación no puede leer los respaldos.

Si los respaldos van en un segundo disco, móntelo en `/var/backups/digaipe` (entrada en `/etc/fstab`) *antes* de los comandos `chown` y `chmod` de arriba.

5.2. Instalar Node.js 24

Los repositorios de Debian y Ubuntu traen versiones de Node.js más antiguas. Use el repositorio de NodeSource, que se actualiza con `apt`:

```
curl -fsSL https://deb.nodesource.com/setup_24.x -o /tmp/nodesource_setup.sh
sudo bash /tmp/nodesource_setup.sh
sudo apt install -y nodejs
```

El script solo agrega el repositorio y su clave; puede revisarlo (`less /tmp/nodesource_setup.sh`) antes de ejecutarlo.

Comprobación

`node -v` debe mostrar `v24.<x>`, y `npx -v`, un número de versión.

5.3. Instalar MySQL 8.4

Debian no trae MySQL (solo MariaDB) y Ubuntu trae MySQL 8.0. Use el repositorio APT oficial de MySQL. Descargue el paquete de configuración más reciente desde `https://dev.mysql.com/downloads/repo/apt/` e instálelo:

```
sudo apt install -y gnupg wget
wget https://dev.mysql.com/get/mysql-apt-config_<versión>_all.deb -O
↪  /tmp/mysql-apt-config.deb
sudo apt install -y /tmp/mysql-apt-config.deb
```

En el menú que aparece, elija *MySQL Server & Cluster* → **mysql-8.4-lts** y luego *Ok*. Si se equivoca, vuelva al menú con `sudo dpkg-reconfigure mysql-apt-config`. Después:

```
sudo apt update
sudo apt install -y mysql-server
```

El instalador pide la contraseña de `root` de MySQL: use `MYSQL_ROOT_PASSWORD`. Si pregunta por el método de autenticación, elija el recomendado (`caching_sha2_password`).

Restrinja MySQL al propio servidor creando `/etc/mysql/mysql.conf.d/digaipe.cnf`:

/etc/mysql/mysql.conf.d/digaipe.cnf

```
[mysqld]
# Solo conexiones desde este servidor: la aplicación y los respaldos.
bind-address = 127.0.0.1
mysqlx = OFF
```

Reinicie y cree la base de datos:

```
sudo systemctl restart mysql
sudo systemctl enable mysql
mysql -h 127.0.0.1 -u root -p -e "CREATE DATABASE IF NOT EXISTS digaipe CHARACTER
↪  SET utf8mb4 COLLATE utf8mb4_0900_ai_ci"
```

Comprobación

```
mysql -h 127.0.0.1 -u root -p -e "SELECT VERSION(); SHOW DATABASES LIKE
↪  'digaipe'"
sudo ss -ltnp | grep mysqld
```

La versión debe empezar por `8.4`, la base `digaipe` debe existir y `mysqld` debe escuchar solo en `127.0.0.1:3306`.

5.4. Instalar la aplicación

Clone el repositorio en `/opt/digaipe/app` (capítulo 3) y construya:

```
cd /opt/digaipe/app
sudo npm ci
sudo npm run build
sudo mkdir -p /opt/digaipe/app/var/uploads
sudo chown -R digaipe:digaipe /opt/digaipe/app/var
sudo chmod 2750 /opt/digaipe/app/var /opt/digaipe/app/var/uploads
```

El bit `2` (*setgid*) hace que las carpetas nuevas de `var/uploads` hereden el grupo `digaipe`, de modo que el usuario de respaldos siempre pueda leerlas.

El código, `node_modules` y `build` quedan propiedad de `root`: el servicio puede leerlos pero no modificarlos. Solo `var` es suyo.

Comprobación

`npm run build` debe terminar sin errores, y `ls -a /opt/digaipe/app` debe mostrar la carpeta `build`. **No** debe haber un archivo `.env`: la configuración va en `/etc/digaipe/`.

Nota

El paquete incluye las dependencias de desarrollo (`drizzle-kit`, `tsx`) a propósito: la instalación las usa para aplicar migraciones, crear las cuentas de MySQL y verificar la auditoría.

5.5. Archivos de configuración

La instalación usa tres archivos, uno por cada cuenta de MySQL, para que cada proceso conozca solo la contraseña que necesita. Créelos con `sudo nano <archivo>` y reemplace los valores `<...>`.

/etc/digaipe/setup.env (instalación, restauración y auditoría: root de MySQL)

```
MYSQL_ROOT_PASSWORD=<MYSQL_ROOT_PASSWORD>
DATABASE_URL=mysql://root:<MYSQL_ROOT_PASSWORD>@127.0.0.1:3306/digaipe
APP_DB_USER=digaipe_app
APP_DB_PASSWORD=<APP_DB_PASSWORD>
BACKUP_DB_USER=digaipe_respaldo
BACKUP_DB_PASSWORD=<BACKUP_DB_PASSWORD>
READONLY_DB_USER=digaipe_lectura
READONLY_DB_PASSWORD=
ORIGIN=https://<DIGAIPE_HOST>
BETTER_AUTH_SECRET=<BETTER_AUTH_SECRET>
BOOTSTRAP_ADMIN_EMAIL=<correo del administrador>
BOOTSTRAP_ADMIN_PASSWORD=<BOOTSTRAP_ADMIN_PASSWORD>
BOOTSTRAP_ADMIN_NAME=Administrador
```

/etc/digaipe/digaipe.env (aplicación)

```
NODE_ENV=production
DATABASE_URL=mysql://digaipe_app:<APP_DB_PASSWORD>@127.0.0.1:3306/digaipe
ORIGIN=https://<DIGAIPE_HOST>
BETTER_AUTH_SECRET=<BETTER_AUTH_SECRET>
# Solo Caddy, en este mismo servidor, puede conectarse a la aplicación.
HOST=127.0.0.1
PORT=3000
# IP real del usuario, tomada de la entrada que añade Caddy (auditoría y límite de
↪  intentos).
ADDRESS_HEADER=X-Forwarded-For
XFF_DEPTH=1
# Las cargas de admisiones (hoja de cálculo + PDF) superan el límite por defecto de
↪  512K.
BODY_SIZE_LIMIT=50M
TZ=America/Caracas
```

/etc/digaipe/backup.env (respaldos)

```
DB_HOST=127.0.0.1
DB_NAME=digaipe
BACKUP_DB_USER=digaipe_respaldo
BACKUP_DB_PASSWORD=<BACKUP_DB_PASSWORD>
BACKUP_ROOT=/var/backups/digaipe
UPLOADS_DIR=/opt/digaipe/app/var/uploads
BACKUP_INTERVAL_DAYS=7
BACKUP_HOUR_UTC=6
BACKUP_KEEP_DAYS=70
BACKUP_KEEP_MIN=10
BACKUP_MIN_FREE_MB=512
```

```
sudo chown root:root /etc/digaipe/setup.env /etc/digaipe/digaipe.env
↪  /etc/digaipe/backup.env
sudo chmod 600 /etc/digaipe/setup.env /etc/digaipe/digaipe.env
↪  /etc/digaipe/backup.env
```

Los servicios leen estos archivos como `root` al arrancar (`systemd`), así que no hace falta que los usuarios de servicio puedan leerlos.

Atención

`ORIGIN` y `BETTER_AUTH_SECRET` deben ser idénticos en `setup.env` y en `digaipe.env`, y `APP_DB_PASSWORD` / `BACKUP_DB_PASSWORD`, idénticas en el archivo de instalación y en el que las usa. Si difieren, la aplicación o los respaldos no podrán conectarse.

5.6. Comandos de administración

Estos tres scripts cargan `setup.env` y ejecutan, como `root` de MySQL, la instalación, la auditoría y la restauración. Hacen lo mismo que los servicios `setup`, `audit` y `restore` del modo Docker.

/usr/local/sbin/digaipe-setup

```
#!/bin/bash
# Migraciones, triggers de auditoría, cuentas de MySQL, administrador inicial y
↪  datos base.
# Se puede repetir sin riesgo. Obligatorio después de restaurar o actualizar.
set -euo pipefail
cd /opt/digaipe/app
set -a; . /etc/digaipe/setup.env; set +a
exec sh scripts/docker/setup.sh
```

/usr/local/sbin/digaipe-audit

```
#!/bin/bash
# Uso: digaipe-audit verify [--anchors <archivo>]
#      digaipe-audit anchor --anchors <archivo> [--note <texto>]
set -euo pipefail
cd /opt/digaipe/app
set -a; . /etc/digaipe/setup.env; set +a
exec npx tsx scripts/audit-ledger.ts "$@"
```

/usr/local/sbin/digaipe-restaurar

```
#!/bin/bash
# Uso: digaipe-restaurar <respaldo> <respaldo>   (el nombre dos veces, como
↪  confirmación)
set -euo pipefail
set -a; . /etc/digaipe/backup.env; . /etc/digaipe/setup.env; set +a
RESTORE_CONFIRM="${2:-}" bash /opt/digaipe/app/deploy/backup/restore.sh "${1:?Uso:
↪  digaipe-restaurar <respaldo> <respaldo>}"
# restore.sh copia los archivos como root; la aplicación necesita poder
↪  escribirlos.
chown -R digaipe:digaipe /opt/digaipe/app/var/uploads
```

```
sudo chmod 700 /usr/local/sbin/digaipe-setup /usr/local/sbin/digaipe-audit
↪  /usr/local/sbin/digaipe-restaurar
```

5.7. Ejecutar la instalación

```
sudo /usr/local/sbin/digaipe-setup
```

Comprobación

La salida debe mostrar, en orden y sin errores: `==> Migraciones`, `==> Triggers de auditoría`, `==> Usuarios de base de datos y permisos` (con líneas `Granted digaipe_app ...` y `Granted digaipe_respaldo ...`), `==> Administrador inicial` y `==> Datos base`.

5.8. Servicio de la aplicación

/etc/systemd/system/digaipe.service

```
[Unit]
Description=DIGAIPE (aplicacion web)
After=network-online.target mysql.service
Wants=network-online.target
Requires=mysql.service

[Service]
Type=simple
User=digaipe
Group=digaipe
WorkingDirectory=/opt/digaipe/app
EnvironmentFile=/etc/digaipe/digaipe.env
ExecStart=/usr/local/bin/node build
Restart=on-failure
RestartSec=5
# Archivos nuevos legibles por el grupo (respaldos), no por otros usuarios.
UMask=0027
NoNewPrivileges=true
PrivateTmp=true
ProtectHome=true
ProtectSystem=strict
ReadWritePaths=/opt/digaipe/app/var

[Install]
WantedBy=multi-user.target
```

`WorkingDirectory` es obligatorio: la aplicación busca la plantilla PDF en `static/` y guarda las cargas en `var/uploads/`, ambas relativas a esa carpeta.

```
sudo systemctl daemon-reload
sudo systemctl enable --now digaipe
```

Comprobación

```
systemctl status digaipe
curl -sI http://127.0.0.1:3000/ | head -n 1
```

El servicio debe estar `active (running)` y `curl` debe responder con una línea `HTTP/1.1`. Si falla, vea `journalctl -u digaipe -n 50`.

5.9. Caddy (HTTPS)

Desde el repositorio oficial de Caddy:

```
sudo apt install -y debian-keyring debian-archive-keyring apt-transport-https curl
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/gpg.key' | sudo gpg
↪  --dearmor -o /usr/share/keyrings/caddy-stable-archive-keyring.gpg
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/debian.deb.txt' | sudo tee
↪  /etc/apt/sources.list.d/caddy-stable.list
sudo apt update
sudo apt install -y caddy
```

El paquete crea el usuario `caddy` y el servicio `caddy`. Reemplace `/etc/caddy/Caddyfile` por:

/etc/caddy/Caddyfile

```
# HTTPS de DIGAIPE: Caddy actúa como autoridad certificadora local (red interna).
{
    local_certs
    # La raíz se instala a mano en las PCs cliente, no en este servidor.
    skip_install_trust
}

# Debe coincidir exactamente con ORIGIN (sin https://).
<DIGAIPE_HOST> {
    tls internal
    reverse_proxy 127.0.0.1:3000
    header -Server
}
```

```
sudo caddy validate --config /etc/caddy/Caddyfile
sudo systemctl enable caddy
sudo systemctl restart caddy
```

La autoridad certificadora queda en `/var/lib/caddy/.local/share/caddy/pki/authorities/local/`. **Esa carpeta debe estar en las copias de seguridad del servidor**: si se pierde, Caddy crea otra autoridad y hay que volver a instalar la raíz en todas las PCs (capítulo 7).

5.10. Servicio de respaldos

/etc/systemd/system/digaipe-respaldo.service

```
[Unit]
Description=DIGAIPE (respaldo semanal)
After=mysql.service
Requires=mysql.service

[Service]
Type=simple
User=digaipe-respaldo
Group=digaipe-respaldo
SupplementaryGroups=digaipe
EnvironmentFile=/etc/digaipe/backup.env
ExecStart=/usr/bin/bash /opt/digaipe/app/deploy/backup/worker.sh
```

```
Restart=always
RestartSec=60
NoNewPrivileges=true
PrivateTmp=true
ProtectHome=true
ProtectSystem=strict
ReadWritePaths=/var/backups/digaipe

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now digaipe-respaldo
```

Comprobación

`journalctl -u digaipe-respaldo -n 20` debe mostrar `Backup worker started`, `Backup due: no backup exists yet` y, al cabo de unos segundos, `Backup finished`. `sudo ls /var/backups/digaipe` debe mostrar una carpeta `digaipe-<fecha>`.

5.11. Cortafuegos

```
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
sudo ufw status
```

Omita la regla 22 si no administra el servidor por SSH. Si el sistema usa `nftables` o `firewalld` en lugar de `ufw`, aplique las mismas reglas con esa herramienta.

5.12. Siguiente paso

Continúe con el certificado HTTPS (capítulo 7) y la verificación (capítulo 8).

5.13. Operación diaria

Tarea	Comando
Ver estado	`systemctl status digaipe caddy mysql digaipe-respaldo`
Ver registros de la aplicación	`journalctl -u digaipe -f`
Reaplicar la instalación	`sudo /usr/local/sbin/digaipe-setup` y luego `sudo systemctl restart digaipe`
Detener	`sudo systemctl stop digaipe digaipe-respaldo`

Tarea	**Comando**
Iniciar	`sudo systemctl start digaipe digaipe-respaldo`

Capítulo 6

Instalación en servidor físico: Windows

Instalación sin Docker en Windows Server 2022/2025 (o Windows 10/11 Pro). MySQL, la aplicación y Caddy quedan como servicios de Windows, y los respaldos, como una tarea programada. Los instaladores se descargan de los sitios oficiales en `C:\digaipe\descargas`.

Ruta	Contenido
`C:\digaipe\app`	Aplicación
`C:\digaipe\app\var\uploads`	Archivos de admisiones cargados
`C:\digaipe\config`	Archivos de configuración con contraseñas (solo administradores)
`C:\digaipe\bin`	Scripts de administración
`C:\digaipe\caddy`	Caddy, su configuración y su autoridad certificadora
`C:\digaipe\logs`	Registros de la aplicación y de los respaldos
`D:\digaipe-respaldos`	Respaldos (en un segundo disco)

Si no tiene un disco `D:`, use otra unidad y cambie la ruta en `backup.env` (sección 6.5).

Identificadores de cuentas

Los comandos `icacls` usan identificadores (SID) en vez de nombres, porque los nombres cambian con el idioma de Windows: `*S-1-5-32-544` = Administradores, `*S-1-5-18` = SYSTEM, `*S-1-5-19` = SERVICIO LOCAL (*LocalService*).

6.1. Preparar el sistema

```
tzutil /s "Venezuela Standard Time"
New-Item -ItemType Directory -Force C:\digaipe\descargas, C:\digaipe\config,
↪  C:\digaipe\bin, C:\digaipe\caddy, C:\digaipe\logs, D:\digaipe-respaldos
icacls C:\digaipe /inheritance:r /grant:r "*S-1-5-32-544:(OI)(CI)F"
↪  "*S-1-5-18:(OI)(CI)F" "*S-1-5-19:(OI)(CI)RX"
icacls C:\digaipe\config /inheritance:r /grant:r "*S-1-5-32-544:(OI)(CI)F"
↪  "*S-1-5-18:(OI)(CI)F"
```

```
icacls C:\digaipe\bin /inheritance:r /grant:r "*S-1-5-32-544:(OI)(CI)F"
↪  "*S-1-5-18:(OI)(CI)F"
icacls C:\digaipe\caddy /inheritance:r /grant:r "*S-1-5-32-544:(OI)(CI)F"
↪  "*S-1-5-18:(OI)(CI)F"
icacls C:\digaipe\logs /grant "*S-1-5-19:(OI)(CI)M"
icacls D:\digaipe-respaldos /inheritance:r /grant:r "*S-1-5-32-544:(OI)(CI)F"
↪  "*S-1-5-18:(OI)(CI)F"
```

Por defecto, una carpeta nueva en `C:\` hereda permisos que dejan a cualquier usuario autenticado modificarla. El primer `icacls` los reemplaza: administradores y sistema con control total, y *LocalService* (la cuenta de la aplicación) solo con lectura. Los siguientes cierran las carpetas con contraseñas y scripts, y dan escritura solo donde hace falta.

6.2. Instalar Node.js 24

Descargue el instalador `node-v24.<x>-x64.msi` de `https://nodejs.org/dist/latest-v24.x/` a `C:\digaipe\descargas` e instálelo:

```
msiexec /i C:\digaipe\descargas\node-v24.<x>-x64.msi /qn
```

Use la serie 24 aunque la página principal de nodejs.org ofrezca una versión más nueva.

Cierre y vuelva a abrir PowerShell para que se actualice la variable `PATH`.

Comprobación

`node -v` debe mostrar `v24.<x>`, y `npx -v`, un número de versión.

6.3. Instalar MySQL 8.4

Descargue *MySQL Community Server 8.4 LTS*, *Windows (x86, 64-bit), MSI Installer*, de `https://dev.mysql.com/downloads/mysql/8.4.html` (no hace falta crear una cuenta: use *No thanks, just start my download*). Ejecute `mysql-8.4.<x>-winx64.msi` (instalación típica). Al terminar se abre **MySQL Configurator**; elija:

- *Config Type*: `Server Computer`. Puerto TCP `3306`.
- **Desmarque** *Open Windows Firewall ports for network access.*
- *Root password*: `MYSQL_ROOT_PASSWORD`. No cree otros usuarios: los crea la instalación de DIGAIPE.
- *Windows Service*: nombre `MySQL84`, inicio automático, cuenta *Standard System Account.*

Restrinja MySQL al propio servidor. Abra `C:\ProgramData\MySQL\MySQL Server 8.4\my.ini` con el Bloc de notas como administrador y, en la sección `[mysqld]`, agregue:

my.ini, sección [mysqld]

```
bind-address=127.0.0.1
mysqlx=OFF
```

Agregue el cliente de MySQL al `PATH` del sistema, reinicie el servicio y cree la base:

```
[Environment]::SetEnvironmentVariable('Path',
↪  [Environment]::GetEnvironmentVariable('Path','Machine') + ';C:\Program
↪  Files\MySQL\MySQL Server 8.4\bin', 'Machine')
Restart-Service MySQL84
& 'C:\Program Files\MySQL\MySQL Server 8.4\bin\mysql.exe' -h 127.0.0.1 -u root -p
↪  -e "CREATE DATABASE IF NOT EXISTS digaipe CHARACTER SET utf8mb4 COLLATE
↪  utf8mb4_0900_ai_ci"
```

Comprobación

```
& 'C:\Program Files\MySQL\MySQL Server 8.4\bin\mysql.exe' -h 127.0.0.1 -u
↪  root -p -e "SELECT VERSION(); SHOW DATABASES LIKE 'digaipe'"
Get-NetTCPConnection -LocalPort 3306 -State Listen
```

La versión debe empezar por `8.4`, la base `digaipe` debe existir y `LocalAddress` debe ser `127.0.0.1`. Si el instalador creó una regla de cortafuegos para MySQL, elimínela (`Get-NetFirewallRule -DisplayName *MySQL*` y `Remove-NetFirewallRule`).

6.4. Instalar la aplicación

Instale Git y clone el repositorio en `C:\digaipe\app` siguiendo el capítulo 3. Git for Windows también aporta `bash` y las herramientas que usan los scripts de respaldo (`gzip`, `sha256sum`, `date`, `df`). Luego construya:

```
Set-Location C:\digaipe\app
npm ci
npm run build
New-Item -ItemType Directory -Force C:\digaipe\app\var\uploads
icacls C:\digaipe\app\var /grant "*S-1-5-19:(OI)(CI)M"
```

El servicio de la aplicación se ejecuta como *LocalService*, que solo puede leer `C:\digaipe\app`; el último comando le permite escribir en `var`.

Comprobación

`npm run build` debe terminar sin errores, y `Get-ChildItem -Force C:\digaipe\app` debe mostrar la carpeta `build`. **No** debe haber un archivo `.env`: la configuración va en `C:\digaipe\config`.

6.5. Archivos de configuración

Igual que en Linux, hay un archivo por cuenta de MySQL. Créelos con el Bloc de notas (`notepad C:\digaipe\config\setup.env`) y reemplace los valores `<...>`.

C:\digaipe\config\setup.env (instalación, restauración y auditoría: root de MySQL)

```
MYSQL_ROOT_PASSWORD=<MYSQL_ROOT_PASSWORD>
DATABASE_URL=mysql://root:<MYSQL_ROOT_PASSWORD>@127.0.0.1:3306/digaipe
APP_DB_USER=digaipe_app
APP_DB_PASSWORD=<APP_DB_PASSWORD>
BACKUP_DB_USER=digaipe_respaldo
BACKUP_DB_PASSWORD=<BACKUP_DB_PASSWORD>
READONLY_DB_USER=digaipe_lectura
READONLY_DB_PASSWORD=
ORIGIN=https://<DIGAIPE_HOST>
BETTER_AUTH_SECRET=<BETTER_AUTH_SECRET>
BOOTSTRAP_ADMIN_EMAIL=<correo del administrador>
BOOTSTRAP_ADMIN_PASSWORD=<BOOTSTRAP_ADMIN_PASSWORD>
BOOTSTRAP_ADMIN_NAME=Administrador
```

C:\digaipe\config\digaipe.env (aplicación)

```
NODE_ENV=production
DATABASE_URL=mysql://digaipe_app:<APP_DB_PASSWORD>@127.0.0.1:3306/digaipe
ORIGIN=https://<DIGAIPE_HOST>
BETTER_AUTH_SECRET=<BETTER_AUTH_SECRET>
HOST=127.0.0.1
PORT=3000
ADDRESS_HEADER=X-Forwarded-For
XFF_DEPTH=1
BODY_SIZE_LIMIT=50M
TZ=America/Caracas
```

C:\digaipe\config\backup.env (respaldos; rutas en formato de Git Bash)

```
DB_HOST=127.0.0.1
DB_NAME=digaipe
BACKUP_DB_USER=digaipe_respaldo
BACKUP_DB_PASSWORD=<BACKUP_DB_PASSWORD>
BACKUP_ROOT=/d/digaipe-respaldos
UPLOADS_DIR=/c/digaipe/app/var/uploads
BACKUP_INTERVAL_DAYS=7
BACKUP_HOUR_UTC=6
BACKUP_KEEP_DAYS=70
BACKUP_KEEP_MIN=10
BACKUP_MIN_FREE_MB=512
```

En `backup.env`, `/d/digaipe-respaldos` es la forma en que `bash` escribe `D:\digaipe-respaldos`.

La aplicación se ejecuta como *LocalService* y necesita leer su propio archivo:

```
icacls C:\digaipe\config\digaipe.env /grant "*S-1-5-19:R"
```

Atención

`ORIGIN` y `BETTER_AUTH_SECRET` deben ser idénticos en `setup.env` y en `digaipe.env`, y cada contraseña, idéntica en todos los archivos donde aparece.

6.6. Scripts de administración

Guarde los scripts de PowerShell **sin acentos** (o como UTF-8 con BOM): Windows PowerShell 5.1 lee mal los archivos UTF-8 sin BOM.

C:\digaipe\bin\entorno.ps1 (carga setup.env; lo usan los dos siguientes)

```
$ErrorActionPreference = 'Stop'
Get-Content 'C:\digaipe\config\setup.env' |
    Where-Object { $_ -match '^[A-Za-z_][A-Za-z0-9_]*=.' } |
    ForEach-Object {
        $name, $value = $_ -split '=', 2
        Set-Item -Path "Env:$name" -Value $value
    }
Set-Location 'C:\digaipe\app'
```

C:\digaipe\bin\digaipe-setup.ps1

```
# Migraciones, triggers de auditoria, cuentas de MySQL, administrador inicial y
↪  datos base.
# Se puede repetir sin riesgo. Obligatorio despues de restaurar o actualizar.
. 'C:\digaipe\bin\entorno.ps1'
function Paso([string]$titulo, [scriptblock]$comando) {
    Write-Host "==> $titulo"
    & $comando
    if ($LASTEXITCODE -ne 0) { throw "Fallo el paso: $titulo" }
}
Paso 'Migraciones' { npx drizzle-kit migrate }
Paso 'Triggers de auditoria' { npx tsx scripts/ensure-audit-triggers.ts }
Paso 'Usuarios de base de datos y permisos' { npx tsx scripts/grant-db-users.ts }
Paso 'Administrador inicial' { npx tsx scripts/bootstrap-admin.ts }
Paso 'Datos base: modalidades' { npx tsx scripts/load_admission_mod.ts }
Paso 'Datos base: tipos OCRE' { npx tsx scripts/load_ocre_types.ts }
```

C:\digaipe\bin\digaipe-audit.ps1

```
# Uso: digaipe-audit.ps1 verify [--anchors <archivo>]
#      digaipe-audit.ps1 anchor --anchors <archivo> [--note <texto>]
. 'C:\digaipe\bin\entorno.ps1'
npx tsx scripts/audit-ledger.ts @args
exit $LASTEXITCODE
```

Para los respaldos, tres scripts de `bash`. Guárdelos con finales de línea LF (en el Bloc de notas de Windows 11: abajo a la derecha, cambie *Windows (CRLF)* por *Unix (LF)*).

C:\digaipe\bin\entorno.sh

```
# Cliente de MySQL y variables de backup.env (sin los CR de Windows).
export PATH="/c/Program Files/MySQL/MySQL Server 8.4/bin:$PATH"
set -a
. <(tr -d '\r' < /c/digaipe/config/backup.env)
set +a
```

C:\digaipe\bin\respaldo.sh (lo ejecuta la tarea programada cada hora)

```
#!/usr/bin/env bash
# Decide una vez si toca respaldo (semanal, a BACKUP_HOUR_UTC) y, si toca, lo hace.
. /c/digaipe/bin/entorno.sh
export WORKER_ONCE=1
bash /c/digaipe/app/deploy/backup/worker.sh >> /c/digaipe/logs/respaldo.log 2>&1
```

C:\digaipe\bin\restaurar.sh

```
#!/usr/bin/env bash
# Uso: restaurar.sh <respaldo> <respaldo>   (el nombre dos veces, como
↪  confirmación)
set -euo pipefail
. /c/digaipe/bin/entorno.sh
set -a; . <(tr -d '\r' < /c/digaipe/config/setup.env); set +a
RESTORE_CONFIRM="${2:-}" bash /c/digaipe/app/deploy/backup/restore.sh "${1:?Uso:
↪  restaurar.sh <respaldo> <respaldo>}"
```

6.7. Ejecutar la instalación

```
powershell -ExecutionPolicy Bypass -File C:\digaipe\bin\digaipe-setup.ps1
```

Comprobación

Deben aparecer los seis pasos `==> ...` sin errores, con líneas `Granted digaipe_app ...` y `Granted digaipe_respaldo ...` en el paso de permisos.

6.8. Servicio de la aplicación

NSSM ejecuta la aplicación como servicio de Windows. Descárguelo, copie `nssm.exe` (carpeta `win64` del ZIP) a `C:\digaipe\bin` y cree el servicio:

```
Invoke-WebRequest -UseBasicParsing https://nssm.cc/release/nssm-2.24.zip -OutFile
↪  C:\digaipe\descargas\nssm-2.24.zip
Expand-Archive C:\digaipe\descargas\nssm-2.24.zip -DestinationPath
↪  C:\digaipe\descargas\nssm
Copy-Item C:\digaipe\descargas\nssm\nssm-2.24\win64\nssm.exe
↪  C:\digaipe\bin\nssm.exe
C:\digaipe\bin\nssm.exe install digaipe "C:\Program Files\nodejs\node.exe"
↪  "--env-file=C:\digaipe\config\digaipe.env build"
C:\digaipe\bin\nssm.exe set digaipe AppDirectory C:\digaipe\app
C:\digaipe\bin\nssm.exe set digaipe DisplayName "DIGAIPE (aplicacion web)"
C:\digaipe\bin\nssm.exe set digaipe Start SERVICE_AUTO_START
C:\digaipe\bin\nssm.exe set digaipe DependOnService MySQL84
C:\digaipe\bin\nssm.exe set digaipe AppStdout C:\digaipe\logs\app.log
C:\digaipe\bin\nssm.exe set digaipe AppStderr C:\digaipe\logs\app.log
C:\digaipe\bin\nssm.exe set digaipe AppRotateFiles 1
C:\digaipe\bin\nssm.exe set digaipe AppRotateBytes 10485760
sc.exe config digaipe obj= "NT AUTHORITY\LocalService"
Start-Service digaipe
```

La cuenta del servicio se asigna con `sc.exe` y no con `nssm set ObjectName`: esta última necesita una contraseña vacía como argumento, y Windows PowerShell 5.1 descarta los argumentos vacíos al llamar a programas externos.

`AppDirectory` es obligatorio: la aplicación busca la plantilla PDF en `static` y guarda las cargas en `var\uploads`, ambas relativas a esa carpeta.

Comprobación

```
Get-Service digaipe
(Invoke-WebRequest -UseBasicParsing -MaximumRedirection 0 -ErrorAction
↪  SilentlyContinue http://127.0.0.1:3000/).StatusCode
```

El servicio debe estar `Running` y la petición debe devolver un código (200 o 3xx). Si falla, revise `C:\digaipe\logs\app.log`.

6.9. Caddy (HTTPS)

Descargue Caddy de sus publicaciones oficiales en GitHub (la misma versión que usa Docker; una 2.x más reciente también sirve):

```
Invoke-WebRequest -UseBasicParsing https://github.com/caddyserver/caddy/releases/
↪  download/v2.11.7/caddy_2.11.7_windows_amd64.zip -OutFile
↪  C:\digaipe\descargas\caddy.zip
Expand-Archive C:\digaipe\descargas\caddy.zip -DestinationPath C:\digaipe\caddy
New-Item -ItemType Directory -Force C:\digaipe\caddy\data
notepad C:\digaipe\caddy\Caddyfile
```

C:\digaipe\caddy\Caddyfile

```
# HTTPS de DIGAIPE: Caddy actúa como autoridad certificadora local (red interna).
{
    local_certs
    skip_install_trust
    # Ubicación fija de la autoridad y los certificados (incluir en las copias de
    ↪  seguridad).
    storage file_system {
        root C:/digaipe/caddy/data
    }
}

# Debe coincidir exactamente con ORIGIN (sin https://).
<DIGAIPE_HOST> {
    tls internal
    reverse_proxy 127.0.0.1:3000
    header -Server
}
```

Guarde el archivo sin extensión (en el diálogo, *Tipo: Todos los archivos*). Luego:

```
C:\digaipe\caddy\caddy.exe validate --config C:\digaipe\caddy\Caddyfile
New-Service -Name caddy -DisplayName "Caddy (HTTPS DIGAIPE)" -StartupType Automatic
↪  -BinaryPathName "C:\digaipe\caddy\caddy.exe run --config
↪  C:\digaipe\caddy\Caddyfile"
sc.exe failure caddy reset= 86400 actions= restart/5000/restart/5000/restart/5000
Start-Service caddy
```

La autoridad certificadora queda en `C:\digaipe\caddy\data\pki\authorities\local`. **Esa carpeta debe estar en las copias de seguridad del servidor**: si se pierde, hay que volver a instalar la raíz en todas las PCs (capítulo 7).

6.10. Tarea programada de respaldos

La tarea se ejecuta cada hora; `worker.sh` con `WORKER_ONCE=1` decide cada vez si toca respaldo (cada 7 días, a las 06:00 UTC, o de inmediato si no hay ninguno o se saltó uno).

```
$accion = New-ScheduledTaskAction -Execute 'C:\Program Files\Git\bin\bash.exe'
↪  -Argument 'C:/digaipe/bin/respaldo.sh'
$cada_hora = New-ScheduledTaskTrigger -Once -At (Get-Date).Date.AddMinutes(5)
↪  -RepetitionInterval (New-TimeSpan -Hours 1)
$cuenta = New-ScheduledTaskPrincipal -UserId 'SYSTEM' -LogonType ServiceAccount
↪  -RunLevel Highest
$ajustes = New-ScheduledTaskSettingsSet -StartWhenAvailable -MultipleInstances
↪  IgnoreNew
Register-ScheduledTask -TaskPath '\DIGAIPE\' -TaskName 'Respaldo' -Action $accion
↪  -Trigger $cada_hora -Principal $cuenta -Settings $ajustes
Start-ScheduledTask -TaskPath '\DIGAIPE\' -TaskName 'Respaldo'
```

Comprobación

Espere un minuto y revise `Get-Content C:\digaipe\logs\respaldo.log -Tail 20`: debe decir `Backup due: no backup exists yet` y `Backup finished`. `Get-ChildItem D:\digaipe-respaldos` debe mostrar una carpeta `digaipe-<fecha>`.

Compruebe la restauración en esta plataforma

Los scripts de respaldo se escribieron para los contenedores Linux; en Windows se ejecutan con el `bash` de Git for Windows. Antes de depender de ellos, haga un simulacro de restauración completo (sección 9.7) en un equipo aparte con el primer respaldo.

Diferencia con Linux y Docker

Si un respaldo falla, se vuelve a intentar en la siguiente hora (en Linux y Docker, a las 6 horas), así que un problema persistente, como un disco lleno, deja un evento `backup.failed` por hora en la auditoría hasta que se corrija.

6.11. Cortafuegos

```
New-NetFirewallRule -DisplayName "DIGAIPE HTTP/HTTPS" -Direction Inbound -Protocol
↪  TCP -LocalPort 80,443 -Action Allow
```

6.12. Siguiente paso

Continúe con el certificado HTTPS (capítulo 7) y la verificación (capítulo 8).

6.13. Operación diaria

Tarea	Comando
Ver estado	`Get-Service MySQL84, digaipe, caddy`
Ver registros de la aplicación	`Get-Content C:\digaipe\logs\app.log -Tail 50 -Wait`
Reaplicar la instalación	`powershell -ExecutionPolicy Bypass -File C:\digaipe\bin\digaipe-setup.ps1` y luego `Restart-Service digaipe`
Detener	`Stop-Service digaipe; Disable-ScheduledTask -TaskPath '\DIGAIPE\' -TaskName Respaldo`
Iniciar	`Start-Service digaipe; Enable-ScheduledTask -TaskPath '\DIGAIPE\' -TaskName Respaldo`

Capítulo 7

Certificado HTTPS en las PCs cliente

Un servidor de la red interna no es accesible desde internet, así que Let's Encrypt no puede emitirle un certificado. Por eso, por defecto, Caddy actúa como **autoridad certificadora local**: emite y renueva el certificado del sitio por su cuenta. (Si el servidor sí es accesible desde internet con un nombre público, vea la sección 7.4: no hace falta nada de este capítulo en las PCs.) Cada PC cliente debe confiar **una vez** en la raíz de esa autoridad; mientras no lo haga, el navegador mostrará *La conexión no es privada*. La raíz es válida por 10 años.

7.1. Exportar la raíz

Abra una vez `https://<DIGAIPE_HOST>` desde el propio servidor o desde una PC (aunque salga el aviso): así Caddy crea la autoridad si aún no existe. Luego copie la raíz:

Modo	Comando
Docker	`docker compose cp caddy: /data/caddy/pki/authorities/local/root.crt ./digaipe-ca.crt` (desde la carpeta `deploy`)
Podman	`podman compose cp caddy: /data/caddy/pki/authorities/local/root.crt ./digaipe-ca.crt` (usuario `digaipe`, desde la carpeta `deploy`)
Linux	`sudo cp /var/lib/caddy/.local/share/caddy/pki/authorities/local/root.crt ./digaipe-ca.crt`
Windows	`Copy-Item C:\digaipe\caddy\data\pki\authorities\local\root.crt .\digaipe-ca.crt`

Solo `root.crt`

Copie únicamente `root.crt`. El archivo `root.key` de la misma carpeta es la clave privada de la autoridad: quien la tenga puede emitir certificados en los que confiarán todas las PCs. Nunca salga del servidor.

Registre la suma SHA-256 del archivo en la hoja de datos (`sha256sum digaipe-ca.crt` o `Get-FileHash digaipe-ca.crt`) y compruébela en cada PC antes de instalarlo. Así se asegura de

que nadie lo reemplazó en el camino.

7.2. Instalar la raíz en las PCs

Windows (Chrome, Edge) En PowerShell como administrador: `certutil -addstore -f Root digaipe-ca.crt`. En un dominio de Active Directory, distribúyalo por directiva de grupo (*Configuración del equipo → Directivas → Configuración de Windows → Configuración de seguridad → Directivas de clave pública → Entidades de certificación raíz de confianza*).

Firefox *Configuración → Privacidad y seguridad → Certificados → Ver certificados → Autoridades → Importar*, y marque *Confiar en esta CA para identificar sitios web*. En PCs gestionadas, puede activar en su lugar la directiva `security.enterprise_roots.enabled`, que hace que Firefox use el almacén de Windows.

Linux (Debian/Ubuntu) `sudo cp digaipe-ca.crt /usr/local/share/ca-certificates/` y `sudo update-ca-certificates`. Firefox y Chrome en Linux pueden necesitar la importación manual como arriba.

Comprobación

Abra `https://<DIGAIPE_HOST>`: debe cargar la página de inicio sin aviso, con el candado cerrado.

7.3. Conservar la autoridad

La carpeta de la autoridad (volumen `digaipe_caddy-data` en Docker; las rutas de la tabla anterior en servidor físico) debe estar en las copias de seguridad del servidor. **Los respaldos de DIGAIPE no la incluyen.** Si se pierde, Caddy crea una autoridad nueva y hay que volver a instalar la raíz en todas las PCs.

7.4. Certificado público (Let's Encrypt)

Si `<DIGAIPE_HOST>` es un **nombre** registrado en el DNS público (no una IP ni un nombre `.local`) y los puertos 80 y 443 del servidor son accesibles desde internet, Caddy puede obtener y renovar solo un certificado de Let's Encrypt, en el que todos los navegadores ya confían. En el `Caddyfile`:

- borre la línea `local_certs` del bloque global, y
- borre la línea `tls internal` del bloque del sitio.

Reinicie Caddy (Docker: `docker compose restart caddy`) y abra el sitio: el primer acceso tarda unos segundos mientras se emite el certificado. Si falla, `docker compose logs caddy` o `journalctl -u caddy` indican el motivo (casi siempre, el puerto 80 no es accesible desde internet).

Exponer el servidor

Esta opción hace que DIGAIPE sea accesible desde internet, no solo desde la red interna. Decídalo con el responsable de seguridad de la institución: un sistema con datos personales expuesto a internet recibe intentos de acceso constantes.

En Docker, `deploy/Caddyfile` es un archivo del repositorio: antes de cada `git pull`, guarde su cambio con `git stash` y recupérelo después con `git stash pop`.

7.5. Usar el certificado de la institución

Si la institución tiene su propia autoridad certificadora, puede usar un certificado emitido por ella y evitar instalar otra raíz en las PCs. Copie el certificado y la clave al servidor y reemplace la línea `tls internal` del `Caddyfile` por:

```
tls /ruta/digaipe.crt /ruta/digaipe.key
```

En Docker, monte además los archivos en el servicio `caddy` de `compose.yaml` (por ejemplo, `./certs:/certs:ro`) y use `tls /certs/digaipe.crt /certs/digaipe.key`. El certificado debe incluir `<DIGAIPE_HOST>` en su *Subject Alternative Name*, y usted deberá renovarlo antes de que venza.

7.6. Cambiar el nombre del servidor

Si cambia `<DIGAIPE_HOST>`, cámbielo en todos los lugares a la vez y reinicie:

- Docker: `DIGAIPE_HOST` y `ORIGIN` en `deploy/.env`; luego `docker compose up -d`.
- Servidor físico: `ORIGIN` en `setup.env` y `digaipe.env`, y el nombre del sitio en el `Caddyfile`; luego reinicie la aplicación y Caddy.

La raíz no cambia, así que las PCs no tienen que reinstalarla. Todas las sesiones abiertas se cierran.

Capítulo 8

Verificación posterior a la instalación

Recorra esta lista al terminar cualquier instalación y anote el resultado en el registro.

#	Comprobación	Resultado esperado
1	Todos los servicios en marcha (Docker: `docker compose ps -a`; Linux: `systemctl status`; Windows: `Get-Service`)	En marcha; `setup` terminado con código 0
2	Desde una PC cliente, abrir `https://<DIGAIPE_HOST>`	Página de inicio sin aviso de certificado
3	Desde una PC cliente, abrir `http://<DIGAIPE_HOST>`	Redirige a `https://`
4	Desde una PC cliente, probar los puertos internos: `Test-NetConnection <DIGAIPE_HOST> -Port 3306` y `-Port 3000`	`TcpTestSucceeded : False` en ambos
5	Iniciar sesión con `BOOTSTRAP_ADMIN_EMAIL`	Entra a DIGAIPE
6	Administración → Auditoría	Aparece el inicio de sesión del punto 5, con la IP de la PC cliente (no `127.0.0.1` ni la IP del servidor)
7	Crear un usuario de prueba con un rol, iniciar sesión con él y comprobar que solo ve lo que permite su rol	Las páginas sin permiso responden *acceso denegado* y quedan en la auditoría como `route.access_denied`
8	Cargar un lote de admisiones de prueba y generar una planilla PDF	La carga y el PDF funcionan
9	Primer respaldo (registros de `backup-worker`, `digaipe-respaldo` o `respaldo.log`)	`Backup finished.`; en Auditoría, un evento `backup.completed`
10	Verificar la cadena de auditoría (capítulo 10)	Verificación correcta, sin triggers faltantes
11	Reiniciar el servidor y repetir 1, 2 y 5	Todo arranca solo
12	Borrar el usuario y los datos de prueba de los puntos 7 y 8, si no deben quedar	—

Punto 6

Si la auditoría muestra siempre la misma IP (la del servidor o `127.0.0.1`), la aplicación no está leyendo la cabecera `X-Forwarded-For`: revise `ADDRESS_HEADER` y `XFF_DEPTH` (capítulo 12). Sin la IP real, el límite de intentos de inicio de sesión bloquearía a todos los usuarios a la vez.
Con **Podman sin root**, una IP interna (`10.x`, `169.254.x`) igual para todas las PCs indica que la red de Podman oculta la dirección del cliente; vea la sección 4.2.6.

Después de verificar

- Cambie o retire `BOOTSTRAP_ADMIN_PASSWORD` del archivo de configuración si su política lo exige: el administrador ya existe y no se vuelve a crear.
- Haga la primera copia del respaldo a un medio externo y el primer anclaje de auditoría.

Capítulo 9

Respaldos y restauración

Este capítulo resume la operación en los tres modos. El procedimiento detallado, con la tabla de errores de restauración, está en `docs/respaldos.md` del repositorio.

9.1. Cómo funcionan

Qué	Valor por defecto
Frecuencia	Cada 7 días (`BACKUP_INTERVAL_DAYS`)
Hora	06:00 UTC = 02:00 en Caracas (`BACKUP_HOUR_UTC`)
Conservación	Se borran los de más de 70 días (`BACKUP_KEEP_DAYS`), pero siempre quedan los 10 más recientes (`BACKUP_KEEP_MIN`)
Espacio mínimo	No empieza con menos de 512 MB libres o el doble del último respaldo (`BACKUP_MIN_FREE_MB`)
Respaldo perdido	Si el servidor estaba apagado, se hace en cuanto vuelve (si el último tiene más de 8 días). Al instalar, se hace uno de inmediato
Bloqueo	Ninguno: instantánea consistente de MySQL; la aplicación sigue funcionando

Cada respaldo es una carpeta `digaipe-<fecha UTC>` con `digaipe.sql.gz` (datos, auditoría y procedimientos, sin triggers), `uploads/` (archivos cargados), `MANIFEST` (fecha, versión de MySQL y cabeza de la cadena de auditoría) y `SHA256SUMS`. Cada respaldo, y cada fallo, queda en la auditoría como `backup.completed` o `backup.failed`.

Atención

No cambie el nombre de las carpetas de respaldo: la antigüedad se lee del nombre, y una carpeta con otro nombre nunca se cuenta ni se borra. Los respaldos contienen datos personales y hashes de contraseñas: guárdelos con el mismo cuidado que la base.

9.2. Revisión semanal (5 minutos)

1. El último resultado debe ser `Backup finished.`:
   - Docker: `docker compose logs --tail 20 backup-worker`

- Linux: `journalctl -u digaipe-respaldo -n 20`
- Windows: `Get-Content C:\digaipe\logs\respaldo.log -Tail 20`

2. En Auditoría, filtrando por acción, debe haber un `backup.completed` de los últimos 7 días y ningún `backup.failed` sin atender.
3. Copie la carpeta del respaldo más reciente a un medio fuera del servidor.

9.3. Respaldo manual

Por ejemplo, antes de una actualización:

Modo	Comando
Docker	`docker compose --profile ops run --rm backup`
Linux	`sudo systemd-run --wait --pipe -p User=digaipe-respaldo -p SupplementaryGroups=digaipe -p EnvironmentFile=/etc/digaipe/backup.env /usr/bin/bash /opt/digaipe/app/deploy/backup/backup.sh`
Windows	`& 'C:\Program Files\Git\bin\bash.exe' -c '. /c/digaipe/bin/entorno.sh && bash /c/digaipe/app/deploy/backup/backup.sh'`

9.4. Antes de restaurar

Restaurar devuelve el sistema al momento del respaldo. Todo lo posterior se pierde de la base activa: admisiones, planillas, revisiones, usuarios y eventos de auditoría de ese periodo. Restaure solo si la base está dañada o se perdió, o por decisión documentada ante un incidente.

1. Registre el incidente: qué pasó, quién autoriza, fecha y hora.
2. Elija el respaldo más reciente anterior al problema y lea su `MANIFEST`.
3. Avise a los usuarios: el sistema estará detenido unos minutos.

9.5. Restaurar

En todos los modos, el orden es el mismo: detener la aplicación y los respaldos, restaurar, **volver a ejecutar la instalación** (reconstruye los triggers de auditoría, que los respaldos no traen), verificar la auditoría y arrancar. Reemplace `digaipe-20261028T060000Z` por el respaldo elegido.

Docker (desde `deploy`)

```
docker compose stop app backup-worker
docker compose --profile ops run --rm -e RESTORE_CONFIRM=digaipe-20261028T060000Z
↪  restore /scripts/restore.sh digaipe-20261028T060000Z
docker compose run --rm setup
docker compose --profile ops run --rm audit verify
docker compose start app backup-worker
```

Linux

```
sudo systemctl stop digaipe digaipe-respaldo
sudo /usr/local/sbin/digaipe-restaurar digaipe-20261028T060000Z
↪  digaipe-20261028T060000Z
sudo /usr/local/sbin/digaipe-setup
sudo /usr/local/sbin/digaipe-audit verify
sudo systemctl start digaipe digaipe-respaldo
```

Windows

```
Stop-Service digaipe
Disable-ScheduledTask -TaskPath '\DIGAIPE\' -TaskName Respaldo
& 'C:\Program Files\Git\bin\bash.exe' C:/digaipe/bin/restaurar.sh
↪  digaipe-20261028T060000Z digaipe-20261028T060000Z
powershell -ExecutionPolicy Bypass -File C:\digaipe\bin\digaipe-setup.ps1
powershell -ExecutionPolicy Bypass -File C:\digaipe\bin\digaipe-audit.ps1 verify
Start-Service digaipe
Enable-ScheduledTask -TaskPath '\DIGAIPE\' -TaskName Respaldo
```

La restauración comprueba `SHA256SUMS`, hace antes un **respaldo de seguridad** del estado actual (su nombre aparece en la salida: anótelo en el incidente y no lo borre), reemplaza la base y los archivos, y comprueba la cabeza de la cadena de auditoría.

Comprobación

Inicie sesión como administrador. En Auditoría deben aparecer `backup.restored` y, justo después, `schema.triggers_repaired`. Fuera de una restauración, un `schema.triggers_repaired` significa que alguien borró o cambió un trigger de auditoría: trátelo como incidente.

9.6. Recuperar en un servidor nuevo

1. Instale el servidor completo según su modo (capítulos 4 a 6) con los **mismos** archivos de configuración si los conserva, pero **sin** ejecutar todavía la instalación ni arrancar la aplicación. En Docker: `docker compose up -d db` (solo la base) y espere a que esté `healthy`.
2. Copie la carpeta del respaldo a la carpeta de respaldos del servidor nuevo.
3. Siga la sección anterior desde el paso de restaurar, agregando `SKIP_SAFETY_BACKUP=1` (no hay nada que respaldar): en Docker con `-e SKIP_SAFETY_BACKUP=1`; en servidor físico, anteponiendo `SKIP_SAFETY_BACKUP=1` al comando en Linux (`sudo SKIP_SAFETY_BACKUP=1 /usr/local/sbin/digaipe-restaurar ...`) o agregando `export SKIP_SAFETY_BACKUP=1` a `restaurar.sh` en Windows (y quitándolo después).
4. Instale la raíz HTTPS nueva en las PCs (capítulo 7), salvo que haya conservado la carpeta de la autoridad.

9.7. Simulacro de restauración

Un respaldo que nunca se restauró no está comprobado. Al menos una vez por periodo académico, haga la sección anterior completa en un equipo aparte (nunca en el servidor de producción) con

el respaldo más reciente, y compruebe que puede iniciar sesión y ver las últimas admisiones. Registre la fecha, el respaldo usado y el resultado.

Capítulo 10

Auditoría: verificar y anclar

El registro de auditoría es una cadena de hashes: cualquier cambio en un evento rompe la cadena. Pero alguien con acceso de administrador a MySQL podría borrar los eventos más recientes o reescribir la cadena completa. Para detectarlo, se **ancla** periódicamente la cabeza de la cadena en un archivo guardado fuera de su alcance: el USB del auditor o una carpeta compartida de otra PC en la que el administrador de la base no pueda escribir.

Cuándo	**Qué**
Diario	Verificar la cadena y las anclas
Semanal y al cerrar cada proceso de admisión	Anclar la cabeza actual
Tras cada restauración, actualización o uso de la cuenta `root` de MySQL	Verificar y guardar la salida con el registro de la intervención

10.1. Comandos

Docker (desde `deploy`; el archivo de anclas está en `ANCHOR_DIR`)

```
docker compose --profile ops run --rm audit
docker compose --profile ops run --rm audit anchor --anchors
↪  /anchors/digaipe-anchors.jsonl --note "cierre UA06426"
```

Linux (con el USB del auditor montado, por ejemplo, en `/media/auditor`)

```
sudo /usr/local/sbin/digaipe-audit verify --anchors
↪  /media/auditor/digaipe-anchors.jsonl
sudo /usr/local/sbin/digaipe-audit anchor --anchors
↪  /media/auditor/digaipe-anchors.jsonl --note "cierre UA06426"
```

Windows (con el USB del auditor, por ejemplo, en `E:`)

```
powershell -ExecutionPolicy Bypass -File C:\digaipe\bin\digaipe-audit.ps1 verify
↪  --anchors E:\digaipe-anchors.jsonl
powershell -ExecutionPolicy Bypass -File C:\digaipe\bin\digaipe-audit.ps1 anchor
↪  --anchors E:\digaipe-anchors.jsonl --note "cierre UA06426"
```

`anchor` verifica primero la cadena y se niega a anclar una cadena rota. Imprime la línea del ancla: cópiela también en un registro independiente (libro de actas, correo interno). Cada ancla queda además en la propia cadena como evento `audit.anchored`.

Si la verificación falla

No repare nada. Conserve la base, los respaldos y el archivo de anclas tal como están y registre el incidente. La salida indica qué eventos no coinciden. Si dice `audit triggers missing`, ejecute la instalación (`setup`) y verifique de nuevo; no deje la aplicación funcionando sin triggers.

Después de una restauración, la verificación con `--anchors` marcará como faltantes las anclas posteriores al respaldo restaurado. Es lo esperado: el evento `backup.restored` y el respaldo de seguridad lo documentan. Guarde esa salida con el registro del incidente.

10.2. Cuenta de solo lectura para auditores

Si los auditores necesitan consultar la base con herramientas SQL, defina `READONLY_DB_PASSWORD` (en `deploy/.env` o en `setup.env`) y vuelva a ejecutar la instalación. La cuenta `digaipe_lectura` lee todas las tablas, incluida la auditoría, pero no ve contraseñas, tokens de sesión ni códigos de verificación, no puede modificar nada y exige conexión cifrada (TLS). Como MySQL no acepta conexiones desde la red, se usa desde el propio servidor:

Modo	Comando
Docker	`docker compose exec db mysql -h 127.0.0.1 --ssl-mode=REQUIRED -u digaipe_lectura -p digaipe`
Linux / Windows	`mysql -h 127.0.0.1 --ssl-mode=REQUIRED -u digaipe_lectura -p digaipe`

Capítulo 11

Actualización

En todos los modos: haga un **respaldo manual** justo antes (capítulo 9), traiga el código nuevo con `git pull`, reconstruya, vuelva a ejecutar la instalación (aplica las migraciones nuevas) y verifique (capítulo 8).

Las migraciones no se deshacen

Una vez aplicadas las migraciones de la versión nueva, volver a la versión anterior del programa no devuelve la base a su estado anterior. Para volver atrás hay que restaurar el respaldo manual hecho antes de actualizar, y se pierde lo hecho desde entonces. Por eso: haga el respaldo, compruebe que terminó bien, anote el commit actual y actualice fuera del horario de uso.

11.1. Docker

Desde `/opt/digaipe/app/deploy` (o `C:\digaipe\app\deploy`; en Windows, sin `sudo`):

```
docker compose --profile ops run --rm backup
sudo git log -1 --format=%h
sudo git pull
docker compose build
docker compose up -d
docker compose logs setup
```

`deploy/.env` no está en Git, así que `git pull` no lo toca. Compare `.env.example` con su `.env` por si la versión nueva trae variables nuevas.

11.2. Servidor físico

Linux

```
sudo systemctl stop digaipe digaipe-respaldo
sudo systemd-run --wait --pipe -p User=digaipe-respaldo -p
↪  SupplementaryGroups=digaipe -p EnvironmentFile=/etc/digaipe/backup.env
↪  /usr/bin/bash /opt/digaipe/app/deploy/backup/backup.sh
```

```
cd /opt/digaipe/app
sudo git log -1 --format=%h
sudo git pull
sudo npm ci
sudo npm run build
sudo /usr/local/sbin/digaipe-setup
sudo systemctl start digaipe digaipe-respaldo
```

Windows

```
Stop-Service digaipe
Disable-ScheduledTask -TaskPath '\DIGAIPE\' -TaskName Respaldo
& 'C:\Program Files\Git\bin\bash.exe' -c '. /c/digaipe/bin/entorno.sh && bash
↪  /c/digaipe/app/deploy/backup/backup.sh'
Set-Location C:\digaipe\app
git log -1 --format=%h
git pull
npm ci
npm run build
powershell -ExecutionPolicy Bypass -File C:\digaipe\bin\digaipe-setup.ps1
Start-Service digaipe
Enable-ScheduledTask -TaskPath '\DIGAIPE\' -TaskName Respaldo
```

`git pull` no toca los archivos cargados (`var/uploads` está excluido de Git) ni la configuración (fuera de la carpeta). Revise si la versión nueva cambió `deploy/Caddyfile` o `deploy/.env.example`, y traslade los cambios a su `Caddyfile` y a sus archivos `.env`. Si cambió la versión requerida de Node.js o de MySQL, actualícela antes.

11.3. Volver a la versión anterior

Si la versión nueva falla y hay que volver atrás:

1. Detenga la aplicación y los respaldos.
2. Vuelva al commit anotado: `git checkout <commit>` (con `sudo` en Linux).
3. Reconstruya: Docker, `docker compose build`; servidor físico, `npm ci` y `npm run build`.
4. Restaure el respaldo manual hecho antes de actualizar (capítulo 9), incluido el paso de volver a ejecutar la instalación.

Cuando corrijan el problema, vuelva a la rama con `git checkout <rama>` antes del siguiente `git pull`.

Capítulo 12

Solución de problemas

Primero mire los registros: `docker compose logs <servicio>` (Docker; `podman compose logs <servicio>` con Podman), `journalctl -u <servicio> -n 100` (Linux) o `C:\digaipe\logs` (Windows).

Síntoma	**Causa probable**	**Qué hacer**
`Defina <VARIABLE> en deploy/.env` al arrancar	Falta un valor obligatorio en `.env`	Complételo y repita `docker compose up -d`
`npm ci` o `docker compose build` falla con errores de red (`ETIMEDOUT`, `ENOTFOUND`, `ECONNRESET`)	El servidor no llega a `registry.npmjs.org` o `cdn.sheetjs.com`, o la red exige un proxy	Compruebe la salida a internet (capítulo 2); configure el proxy y repita
`git` responde *detected dubious ownership*	La copia pertenece a otro usuario (en Linux, a `root`)	En Linux, use `sudo git`. En Windows, ejecute Git desde la misma cuenta de administrador que clonó
`setup` falla con `Access denied for user 'root'`	La contraseña de root no coincide. En Docker, la base se inicializó con otra `MYSQL_ROOT_PASSWORD` (solo se fija la primera vez)	Use la contraseña original. Cambiarla en `.env` no la cambia en una base ya creada
La aplicación no arranca: `Access denied for user 'digaipe_app'`	`APP_DB_PASSWORD` distinta entre la instalación y la aplicación, o no se ejecutó la instalación	Iguale las contraseñas y vuelva a ejecutar la instalación
`DATABASE_URL is not set`	La aplicación arrancó sin su archivo de configuración	Revise `EnvironmentFile` (Linux) o el argumento `--env-file` de NSSM (Windows)
Error de URL o de conexión con una contraseña correcta	La contraseña tiene caracteres que no son letras o números	Genere otra (sección 2.5)
La conexión no es privada	La PC no confía en la raíz, o el nombre usado no es `<DIGAIPE_HOST>`	Instale la raíz (capítulo 7); abra el sistema exactamente con el nombre configurado

Síntoma	Causa probable	Qué hacer
Error 403 *Cross-site POST form submissions are forbidden* al iniciar sesión o enviar un formulario	`ORIGIN` no coincide con la dirección de la barra del navegador	Ponga `ORIGIN=https://<DIGAIPE_HOST>` exacto, sin barra final ni puerto, y reinicie
La carga de admisiones falla con error 413	Archivos mayores que `BODY_SIZE_LIMIT`	Aumente el valor (por defecto `50M`) y reinicie la aplicación
La auditoría muestra siempre la misma IP	Falta `ADDRESS_HEADER=X-Forwarded-For` o `XFF_DEPTH=1`	Agréguelos y reinicie. No los use si la aplicación es accesible sin pasar por Caddy
La planilla PDF falla: `ENOENT` en `static/digaipe_form.pdf`	La aplicación no se ejecuta desde su carpeta	Revise `WorkingDirectory` (Linux) o `AppDirectory` (Windows)
Error al guardar cargas: `EACCES` o `EPERM` en `var/uploads`	La cuenta del servicio no puede escribir allí (p. ej., después de una restauración en Linux)	Linux: `sudo chown -R digaipe:digaipe /opt/digaipe/app/var`. Windows: repita el `icacls` de `var`
`$'\r': command not found`	Un script `.sh` tiene finales de línea CRLF	Prepare el paquete de nuevo con `.gitattributes` (capítulo 3), o convierta el archivo a LF
`Backup FAILED at step: free space`	Poco espacio en la carpeta de respaldos	Libere espacio o use un disco mayor. No borre respaldos a mano sin copiarlos antes
`Backup FAILED at step: mysqldump`	La cuenta de respaldos no tiene permisos o la contraseña no coincide	Vuelva a ejecutar la instalación (recrea los permisos) y compare `BACKUP_DB_PASSWORD`
`verify` informa `audit triggers missing`	No se ejecutó la instalación después de restaurar, o alguien borró triggers	Ejecute la instalación. Si no hubo restauración, trátelo como incidente
Docker Desktop: DIGAIPE no responde tras reiniciar el servidor	Nadie inició sesión en Windows	Vea la sección 4.1.2
Podman: `caddy` no arranca, *rootlessport cannot expose privileged port 80*	Falta permitir puertos bajos a usuarios normales	Aplique `net.ipv4.ip_unprivileged_port_start=80` (sección 4.2)
Podman: `setup` y `app` nunca arrancan; `db` queda en `starting`	Los healthchecks no se ejecutan: la sesión no tiene systemd de usuario	Entre con `sudo machinectl shell digaipe@`, no con `sudo -u` ni `su`
Podman: *Cannot connect to the Docker daemon* o *no such file* en `podman.sock`	El socket de Podman no está activo para ese usuario	`systemctl --user enable --now podman.socket`

Síntoma	**Causa probable**	**Qué hacer**
Podman: *permission denied* al montar `BACKUP_DIR`, `ANCHOR_DIR` o `.env` ilegible	La carpeta o el archivo no pertenecen al usuario `digaipe`, o SELinux bloquea el montaje	`chown digaipe:digaipe`; en discos sin etiquetas SELinux, vea la sección 4.2
Podman: DIGAIPE no vuelve tras reiniciar el servidor	Falta `enable-linger` o el servicio de usuario `digaipe.service`	Sección 4.2.5
Podman: la auditoría muestra la misma IP interna para todas las PCs	La red de Podman sin root oculta la IP del cliente	Sección 4.2.6

Apéndice A

Referencia

A.1. Variables de configuración

Variable	Usada por	Descripción
`MYSQL_ROOT_PASSWORD`	instalación, restauración	Contraseña de root de MySQL
`DATABASE_URL`	aplicación, instalación	URL de conexión. En Docker se compone sola; en servidor físico se escribe en `setup.env` (root) y `digaipe.env` (`digaipe_app`)
`APP_DB_USER` / `APP_DB_PASSWORD`	instalación, aplicación	Cuenta de la aplicación (`digaipe_app`)
`BACKUP_DB_USER` / `BACKUP_DB_PASSWORD`	instalación, respaldos	Cuenta de respaldos (`digaipe_respaldo`)
`READONLY_DB_USER` / `READONLY_DB_PASSWORD`	instalación	Cuenta de solo lectura (`digaipe_lectura`); se crea solo si hay contraseña
`DIGAIPE_HOST`	Caddy (Docker)	Nombre o IP del sitio. En servidor físico se escribe directamente en el `Caddyfile`
`ORIGIN`	aplicación, instalación	`https://<DIGAIPE_HOST>`
`BETTER_AUTH_SECRET`	aplicación, instalación	Secreto de sesiones (32+ caracteres). Cambiarlo cierra todas las sesiones
`BOOTSTRAP_ADMIN_EMAIL`, `_PASSWORD`, `_NAME`	instalación	Primer administrador; se crea solo si no existe
`DIGAIPE_VERSION`	Docker	Etiqueta de las imágenes (`local`)
`HOST` / `PORT`	aplicación (físico)	Dirección de escucha: `127.0.0.1` / `3000`
`ADDRESS_HEADER` / `XFF_DEPTH`	aplicación	`X-Forwarded-For` / `1`: IP real del cliente detrás de Caddy
`BODY_SIZE_LIMIT`	aplicación	Tamaño máximo de una carga (`50M`)
`TZ`	aplicación	`America/Caracas`
`BACKUP_DIR`	Docker	Carpeta de respaldos en el servidor (`./backups`)

Variable	Usada por	Descripción
BACKUP_ROOT	respaldos (físico)	Carpeta de respaldos
UPLOADS_DIR	respaldos (físico)	Carpeta var/uploads de la aplicación
DB_HOST / DB_NAME	respaldos (físico)	127.0.0.1 / digaipe
BACKUP_INTERVAL_DAYS	respaldos	Días entre respaldos (7)
BACKUP_HOUR_UTC	respaldos	Hora UTC del respaldo (6 = 02:00 Caracas)
BACKUP_KEEP_DAYS / BACKUP_KEEP_MIN	respaldos	Conservación (70 días / 10 respaldos mínimo)
BACKUP_MIN_FREE_MB	respaldos	Espacio libre mínimo (512)
ANCHOR_DIR	Docker	Carpeta del archivo de anclas de auditoría

A.2. Ubicaciones

Qué	Docker	Linux	Windows
Configuración	deploy/.env	/etc/digaipe/*.env	C:\digaipe\config*.env
Código	/opt/digaipe/app o C:\digaipe\app (imagen digaipe/app)	/opt/digaipe/app	C:\digaipe\app
Base de datos	volumen digaipe_db-data	/var/lib/mysql	C:\ProgramData\MySQL\MySQL Server 8.4\Data
Archivos cargados	volumen digaipe_uploads	/opt/digaipe/app/var/uploads	C:\digaipe\app\var\uploads
Respaldos	BACKUP_DIR	/var/backups/digaipe	D:\digaipe-respaldos
Autoridad HTTPS	volumen digaipe_caddy-data	/var/lib/caddy/.local/share/caddy	C:\digaipe\caddy\data
Registros	docker compose logs	journalctl	C:\digaipe\logs

A.3. Cuentas de MySQL

Cuenta	Para	Puede
root	Instalación, restauración, verificación de auditoría	Todo
digaipe_app	La aplicación	Leer y escribir datos de negocio; solo leer y anexar a la auditoría
digaipe_respaldo	Respaldos	Leer todo para respaldar; anotar eventos backup.*. No modifica datos ni triggers

Cuenta	Para	Puede
`digaipe_lectura`	Auditores (opcional)	Leer todo salvo contraseñas, tokens y códigos; exige TLS

La instalación crea y ajusta estas cuentas cada vez que se ejecuta (`scripts/grant-db-users.ts`). Cada uso de `root` fuera de la instalación debe quedar en el registro de operaciones, seguido de una verificación de auditoría.

A.4. Roles de la aplicación

Cada usuario tiene uno o más roles (Administración → Usuarios y roles). Un usuario sin roles solo ve la página de inicio.

Rol	Puede
Administrador	Todo, incluidos usuarios y auditoría
Generación de planillas	Buscar admisiones, editar datos del estudiante, generar y reimprimir planillas
Matrícula y devolución	Revisar documentos: matricular, devolver y desbloquear (con motivo)
Estadísticas	Ver estadísticas y exportar CSV/PDF
Carga de admisiones	Cargar lotes, gestionar procesos de admisión, importar carreras
Auditor (solo lectura)	Ver todas las páginas, exportaciones y la auditoría; no modifica nada